ragent Research Manual

This guide explains how to use ragent's built-in research system to perform extensive, structured information gathering and report synthesis. The research system decomposes a topic into sub-queries, searches the web and your local project in parallel, gathers and vaults sources, then runs an adversarial quality pipeline before writing a structured dated report (`RESEARCH-<YYYY-MM-DD>.md`).

The same commands are available in three surfaces:

- **TUI** — `/research` slash commands in the interactive terminal interface
- **CLI** — `ragent research <subcommand>` from the shell
- **HTTP API** — `GET/POST/PUT/DELETE /research` endpoints on the ragent server

This document is written for the terminal interface but covers all three.

> **Scope:** `/research` slash commands, the Hyperresearch adversarial pipeline, source vaulting, local/spec cross-referencing, open-access recovery, templates, and the built-in markdown viewer. For web-search engine configuration and diagnostics, see `/websearch` in the TUI (`/websearch help`).

---

1. Purpose and capabilities

The research system is designed for situations where you need more than a single web search — when you need a **structured, cited, traceable report** that synthesises multiple sources, flags contradictions, and stands up to adversarial quality review.

What it does

- **Topic decomposition** — breaks a research topic into focused sub-queries using either a deterministic heuristic planner or an LLM-backed planner.
- **Multi-engine web search** — queries OpenAlex (scholarly works) and Wikipedia in parallel, plus optional API-backed engines (LangSearch, Tavily, Perplexity, Exa) when configured.
- **Local project cross-referencing** — scans your codebase and project files for relevant content, scoring matches by keyword overlap with the topic.
- **Prior-spec cross-referencing** — reads existing specifications under `specs/<id>/` and weaves them into the research corpus.
- **Source vaulting** — persists every captured source (body text + metadata) to a SQLite-indexed vault so future runs can reuse sources without re-fetching.
- **Adversarial quality pipeline** — the `full` and `dissertation` tiers run a 16-step deterministic pipeline including contradiction detection, loci analysis, corpus critique, gap-fill fetching, triple-draft synthesis, and citation verification.
- **Multiple output formats** — report (default), executive summary, comparison table, source bibliography, and IMRaD scientific format.
- **Diff decks** — `/research deck <name>` turns two of an item's dated reports into a Markdown slide deck showing what changed between them - one slide per concept, each carrying the concept's definition with the findings that belong to it grouped beneath it, and each finding carrying a one-paragraph summary of its observation and analysis (the grouping and the paragraphs written by the selected model when one is wired in, and computed deterministically when not)
  - with every item tagged `NEW`, `ADDED`, `UPDATE` or `OLD`. The deck's structure, counts and tags are deterministic and model-free; only the finding paragraphs and the concept grouping consult

the model, and both fall back to a deterministic equivalent, so a run with no provider produces byte-identical output for the same pair.

- **Open-access recovery** — for short scholarly sources, queries Unpaywall and Europe PMC for legal full-text copies.
- **Seed sources** — start research from a URL, a local file (PDF, DOCX, ODT, markdown, etc.), or an explicit topic string.
- **Resumability** — the source vault and run manifest allow subsequent runs with the same name to skip already-completed gathering.

What it does not do

- It does not replace interactive coding tasks — it produces research documents, not code changes.
- It does not crawl an entire website (use `mf_crawl` for that); it fetches individual pages returned by search engines.
- It does not perform real-time monitoring; each `/research create` is a one-shot gathering session.

---

2. Quick start

Open ragent and run:

```
/research create rust-async "Rust async/await patterns"
```

The TUI will:

1. Create a new research item under `research/rust-async/`.
2. Decompose the topic into sub-queries, search the web (OpenAlex, Wikipedia, plus any configured API engines), optionally scan local project files and specs, and synthesize a structured report.
3. Stream progress into the message window and status bar.
4. Write the final document to `research/rust-async/RESEARCH-<YYYY-MM-DD>.md`.

When it finishes, open the result with:

```
/research open rust-async
```

A typical report (`RESEARCH-<YYYY-MM-DD>.md`) contains:

- YAML frontmatter (title, topic, status, timestamps, source counts, tier, format, open-access disclosure)
- Executive summary
- Top implications
- Open questions
- Findings (with inline `[#N]` citations)
- Findings relationship diagram
- Cross-references to local files and specs
- Contradiction graph (full/dissertation tier)
- Source tensions (full/dissertation tier)
- Numbered references index

---

3. What research items are

A research item is a self-contained folder under `research/<name>/`:

```
research/
   INDEX.md
   rust-async/
       RESEARCH-2026-02-14.md   # report of the run dated 2026-02-14
       RESEARCH-2026-03-02.md   # report of a later `/research update` run
       CORPA-2026-02-14.md      # QA companion for the first run
       CORPA-2026-03-02.md      # QA companion for the later run
       DECK-2026-03-05.md       # `/research deck` output, generated 2026-03-05
       sources/
           web-01.md
           web-02.md
           web-03.md
           local-01.md
           spec-01.md
```

- `RESEARCH-<YYYY-MM-DD>.md` is the deliverable. It contains an executive summary, findings, implications, diagrams, cross-references, open questions, and a numbered references index.
- The report name carries the run's ISO date, so a re-run **adds** a file instead of replacing one: `/research update rust-async` on 2026-03-02 writes `RESEARCH-2026-03-02.md` and leaves `RESEARCH-2026-02-14.md` on disk. The **primary** report is the one named for the item's creation date, which is the file every read surface resolves by default; the newest dated file wins when the primary is absent. An item created by an older ragent keeps its undated `RESEARCH.md` / `CORPA.md`, which resolves by the same rule.
- `sources/` holds the captured body text of every web, local, and spec source so each citation can be traced back to the original text.
- `research/INDEX.md` is a derived table of all items, regenerated automatically on every create/delete/archive.
- `.ragent/research_vault/<run_tag>/` holds the SQLite-indexed source vault with raw content files for resumability.

Naming rules

Research names must be 3–64 characters, start with a lowercase ASCII letter, and contain only lowercase letters, digits, and hyphens. This rule prevents path traversal and keeps directory names URL-safe.

Valid names: `rust-async`, `openai-reasoning`, `q3-review`, `vector-db-2024`

Invalid names: `Rust-Async` (uppercase), `rust_async` (underscore), `1topic` (starts with digit), `a` (too short), `a..b` (dots not allowed)

---

4. The `/research` slash command family

All commands are entered in the TUI input box. The CLI equivalents use `ragent research <subcommand>`.

4.1 `/research help`

Show the command syntax, flags, and formats. Use this whenever you forget a flag.

```
/research help
```

4.2 `/research create`

Run a research session and write a dated report (`RESEARCH-<YYYY-MM-DD>.md`). This is the primary command.

```
/research create <name> [topic]
  [--from-url <URL>] [--from-file <PATH>]
  [--iterations N] [--depth shallow|standard|deep]
  [--tier light|full|dissertation]
  [--mode normal|supervisor|competitive]
  [--format report|executive-summary|comparison-table|source-bibliography|imrad|newsletter]
  [--sources-dir <path>] [--template <name>]
  [--fetch-concurrently N] [--local-concurrently N]
  [--fetch-timeout-secs N]
  [--web-time N] [--web-phase-timeout-secs N] [--local-phase-timeout-secs N]
  [--search-max-retries N] [--search-retry-base-delay-ms N]
  [--max-web-results N] [--max-search-calls N]
  [--max-queries N]
  [--max-concepts N] [--max-findings N]
  [--use-local] [--use-specs] [--use-low-relevance] [--papers] [--use-pdf]
  [--oa-enable] [--no-oa] [--url-cloak]
```

`--mode competitive` runs the multi-researcher competitive pipeline and implicitly defaults `--format` to `comparison-table`, so it need not be passed explicitly.

If the first argument after `/research create` is not a recognised subcommand, the parser treats the whole line as `<name> <topic>`; this lets you type quickly when the name is valid.

Re-running a research item: `/research update <name>` replays the item's recorded invocation line and re-runs the full pipeline, writing a **new dated report** (`RESEARCH-<run date>.md` plus its `CORPA-<run date>.md` companion) and refreshing `sources/` and the index with freshly gathered results. Because the report name carries the run date, the previous run's report is left in place - a same-day re-run still produces its own file. The original invocation is preserved on the replayed run. The same replay is available from the CLI (`ragent research update <name>`) and over HTTP (`PUT /research/{name}`).

Which file does a read command open? Every read surface resolves the report rather than assuming a name. `ResearchIo::resolve_research_md` checks the canonical creation-dated file first (one `stat`), then scans the item directory preferring the lexicographically greatest dated name - the ISO prefix sorts chronologically, so that is the most recent run - and finally accepts the legacy undated `RESEARCH.md`. `/research open`, `/research search`, `/research show`, the index refresh and the CLI all go through that resolver, so an item still opens correctly whether it predates the dated scheme or is several runs old.

Quick syntax note: The `--from-url` flag can be repeated to seed multiple pages. The `--from-file` flag accepts PDF, TXT, and MD files. When neither a topic nor a seed source is supplied, the command errors.

Examples Basic research with default settings (full tier, standard depth, report format):

```
/research create rust-async "Rust async/await patterns"
```

Deep research with full adversarial pipeline:

```
/research create openai-reasoning "OpenAI o1-style reasoning" --tier full --depth deep
```

Quick light-tier lookup for a fast answer:

```
/research create quick-check "Is Rust 1.85 stable" --tier light
```

Local-only research (no web search, scan project files only):

```
/research create local-only "Project error handling" --use-local --papers
```

Seed from a web page (topic derived from page content):

```
/research create from-page --from-url https://example.com/article
```

Seed from multiple web pages:

```
/research create multi-source --from-url https://example.com/a --from-url https://example.com/b
```

Seed from a local document:

```
/research create from-doc --from-file docs/design.md --use-local
```

Comparison table format for evaluating options:

```
/research create benchmark "Comparison of vector databases" --format comparison-table
```

IMRaD scientific report format:

```
/research create paper-review "Attention Is All You Need" --format imrad --tier full
```

Dissertation tier for long-form deliverable with chapter partitioning:

```
/research create thesis "Survey of multi-agent orchestration frameworks" --tier dissertation --de
```

Include project files and specs in the research:

```
/research create rust-async "Rust async patterns" --use-local --use-specs
```

Allow PDF web sources (skipped by default):

```
/research create pdf-research "Machine learning benchmarks" --use-pdf
```

Keep low-relevance sources that would normally be filtered:

```
/research create broad-scan "Emerging programming languages 2024" --use-low-relevance
```

Customise fetch concurrency and timeouts for slow networks:

```
/research create slow-net "Distributed consensus algorithms" --fetch-concurrently 4 --fetch-timeo
```

Use a custom template:

```
/research create quarterly "Q3 performance review" --template quarterly
```

Override iterations explicitly (activates iterative planner/critic loop):

```
/research create deep-iter "Rust memory safety" --iterations 7 --depth deep
```

4.3 `/research list`

List research items, newest first.

```
/research list
/research list --all   # include archived items
```

The default output is a fixed-width `NAME/TITLE/STATUS/CREATED/MODIFIED` table; JSON is available behind a `--json` flag (CLI and HTTP).

4.4 `/research show <name>`

Print metadata for one item: title, status, timestamps, topic, and all captured sources with their provenance (type, URL/path, title, capture time, and open-access recovery note if applicable).

```
/research show rust-async
```

4.5 `/research open <name>`

Open the item's resolved report in the TUI's markdown overlay. See section 11 for viewer controls.

```
/research open rust-async
```

4.6 Concepts section in `/research create`

When `/research create` runs with an LLM configured, a concept-extraction step runs automatically after synthesis. The gathered corpus is sent to the model as a context-bounded payload (each source block headed `--- [#N] Title ---`, where `N` is the source's position in the References Index), and the numbered concept list it returns is embedded in the report as a `## Concepts` section directly above `## Findings`:

```
## Concepts

### 1. Semantic Scholarly Search

**Definition:** Retrieval that understands research questions rather than
exact keywords.

Semantic search ranks by meaning, so a query phrased as a question still
reaches the passages that answer it. The index itself is unchanged - only
the scoring function differs.

It depends on a dense embedding model and is bounded by whatever that model
was trained on, which is why hybrid retrieval layers a keyword pass on top.

**Key Evidence:**
- hybrid retrieval across 200M+ papers ([#6])
```

A `**Definition:**` is three separate paragraphs (roughly 120-200 words in total), not one: the concept and why it matters, then how it works and the distinctions it carries, then where it sits in the wider context. The prompt asks for the shape twice - inline on the field description and again as a writing rule - because a single mention was routinely collapsed into one paragraph.

`[#N]` citations resolve against the References Index at the bottom of the report, exactly like the Findings citations. The section is omitted when no LLM is configured, the extraction fails, or the model returns no concept sections; the step is visible in the progress output as a `concepts` step (started / completed / skipped / failed).

Concept and finding limits. Both the `## Concepts` list and the `## Findings` list are capped by run-level limits: the concept limit defaults to **5** and the finding limit to **20**. Before the cap is applied, each list is reordered most-relevant-first (a finding's relevance is the highest `[#N]` source rank it cites; ties break by cited count, then by original model order) so truncation always discards the least-relevant entries. Override the limits per run with `--max-concepts N` and `--max-findings N`, or persistently with the `research.max_concepts` / `research.max_findings` keys in `ragent.json` (the per-run flags take precedence). A value of `0` means unbounded, matching the `0 disables the cap` convention used elsewhere.

```
/research create brief "Vector databases" --max-concepts 3 --max-findings 10
/research create exhaustive "Multi-agent orchestration" --max-concepts 0 --max-findings 0
```

Cloaking source URLs. Pass `--url-cloak` to write the report's source URLs as defanged plain text instead of clickable links. The transformation rewrites the scheme (`https://` -> `hxxps://`, `http://` -> `hxxp://`), brackets every dot (`example.com` -> `example[.]com`), and wraps the result in a Markdown code span. It applies to the `**Sources:**` bullets under each finding and to the `References Index` table in the report (plus the `Sources Reference` table in the `CORPA` companion); non-URL rows (local paths, spec ids) are untouched, and every other section is unchanged. Use it when the document is ingested by tooling that rejects or flags live URLs. The flag is off by default and is recorded in frontmatter so `/research update` replays it.

```
/research create vendors "Managed vector database vendors" --url-cloak
```

4.7 `/research cluster <name>`

Extract up to 20 core concepts from the captured sources of an existing research item and write them to `research/<name>/CONCEPTS.md`. The command reads every `web-*.md` and `local-*.md` source in the item's `sources/` directory, builds a context-window-aware payload, and asks the active model for a structured concept list. Sources are truncated individually if the combined payload exceeds the model's context budget.

```
/research cluster rust-async
```

The output is a markdown file with a `# Concepts` heading, followed by one numbered section per concept (`## 1. Concept Name`, `## 2. Next Concept`, …) containing a short definition and key evidence bullets. Inline citations use the same `[#N]` style as the report; each concept that cites captured sources gains a `**WebSources:**` block listing those sources in the same format as the `**Sources:**` lists under `## Findings` in the report:

```
## 3. Semantic Scholarly Search

**Definition:** Retrieval that understands research questions rather than
exact keywords.

**Key Evidence:**
- hybrid retrieval across 200M+ papers ([#6])
```

```
**WebSources:**
- [6] Paperguide: AI Search - [https://example.com/...](https://example.com/...) (published 2025
```

The companion is rewritten on every run and uses the same source indices as the report. Concept headings are renumbered sequentially on write, even if the model emits them out of order or unnumbered.

4.8 `/research search <query>`

Full-text search across every item's resolved report. Returns matching item names, titles, and snippets. The search returns up to 25 results.

```
/research search "vector database"
/research search async runtime tokio
```

4.9 `/research delete <name> --yes`

Permanently delete a research item and its `sources/` directory. The `--yes` flag is required; without it the command prompts and refuses.

```
/research delete rust-async --yes
```

4.10 `/research archive <name>`

Mark an item as `archived`. Archived items are hidden from default `list` output but kept on disk. Use `/research list --all` to see archived items.

```
/research archive rust-async
```

4.11 `/research continue <name> [message]`

Resume an in-progress item. Loads state and appends a follow-up sub-question to the plan. The optional message is added to the plan as a new sub-question. The saved state file is written back unchanged when there is no follow-up — an item's `InProgress`/`Complete` status lives in the report's frontmatter, not the state file.

```
/research continue rust-async "Also cover tokio vs async-std comparison"
```

4.12 `/research deck <name>` - the diff deck

Turn two of an item's reports into a single slide deck that tells the *change narrative*: what appeared, what was re-sourced, what changed, and what stayed put between two runs. This is the answer to "I re-ran this research two months later - what actually moved?"

```
/research deck rust-async
ragent research deck rust-async
```

The deck's structure - the cover, the element slides, the status tags, the counts and the summary - is **deterministic and model-free**: it is a pure function of the two input reports, so the same pair always yields byte-identical output and the run works with no provider configured.

Two things on a slide are supplied by the LLM enrichment stage when a provider and model are selected (they are the only part of a deck a model can influence, and the run degrades to a deterministic equivalent without one):

1. **A paragraph under each finding.** The report writes every finding as a headline followed by `**Observation:**` and `**Analysis:**`; the deck condenses both into one paragraph and renders it under the headline. With no model wired in, the paragraph is the first sentence of the observation followed by the first sentence of the analysis, so a model-free deck still explains every finding. The `**Sources:**` list and the source-date note are never turned into prose.
2. **Which findings belong to each concept.** The report's `## Concepts` are written before its findings and do not name them, so the deck groups each finding under the concept it belongs to, at most three per concept. With no model, the grouping is a term-overlap score (a headline match counts double, and a finding must share at least two content terms with a concept to be grouped), so an unrelated finding is never filed under a concept.

Watching a deck run A deck run narrates its stages, so a slow run is visible while it works rather than only when it finishes. The TUI folds the reports into one in-place `Research Progress` message for the run, and the status bar names the stage in flight with its unit count:

```
[research] Research Progress - `rust-async`
Command: /research deck

  [ok] select           - comparing 2026-02-14 to 2026-03-02
  [ok] read             - read 32 item(s) from the older and 35 from the newer report
  [ok] diff             - 3 element(s) in the diff
  [ok] destination      - writing to research/rust-async/DECK-2026-03-05.md
  >    enrich findings  - [7/22] Runtime enforcement beats paper approvals...
  o    group concepts    - no model selected; concept grouping uses the term-overlap ranking
```

- `>` is a stage in flight, `[ok]` a stage that finished, and `o` a stage that did not apply to this run (`no model selected`, `no presentation capability connected`) - so "not applicable" never looks like "still running".
- `enrich findings` reports one line per finding paragraph and `export` reports one line per presentation slide, each carrying `[completed/total]`; a run with no provider skips both enrichment stages and writes the deterministic deck.
- A refusal stops at the stage it failed in and reports no later stage, so you can see where a run gave up.
- The CLI prints the same stages on **stderr**, so the machine-readable JSON line on stdout stays a single parseable record for a piped consumer.

Which two reports By default the deck covers the **two most recent dated reports** of the item: `/research deck rust-async` compares the newest `RESEARCH-<date>.md` against the one before it, rendering the older as the baseline and the newer as current.

To pick the pair yourself, name the date stamps:

```
/research deck rust-async --from 2026-02-14 --to 2026-03-02
```

Each value is a `YYYY-MM-DD` stamp that must match a `RESEARCH-<stamp>.md` in the item directory. Supplying only one pairs it with its **adjacent** dated report - the next older for `--to`, the next newer for `--from`:

```
/research deck rust-async --to 2026-03-02      # 2026-03-02 vs the report before it
/research deck rust-async --from 2026-02-14    # 2026-02-14 vs the report after it
```

An explicit pair must be ordered older-then-newer; `--from 2026-03-02 --to 2026-02-14` is refused rather than silently swapped, because a deck always compares a baseline against a later report.

What a deck covers: the coverage window The **coverage window** is the pair of report dates the deck was built from, shown on the cover slide as `**Coverage:** <older-date> to <newer-date>`. It is the one line that says which two runs you are looking at, so a deck found on disk months later is self-describing.

An item created by an older ragent may still hold an undated `RESEARCH.md`. That legacy file is treated as the **oldest report of the series** - usable as a baseline, never as the current side - and its side of the cover window reads `legacy`. An item whose *only* report is `RESEARCH.md` cannot produce a deck at all, and the refusal says so specifically: it needs its first *dated* report, not a second one.

Status tags Every item on every slide carries exactly one tag from a closed set of four:

Tag	Meaning
`NEW`	The item exists only in the newer report. A reworded finding headline counts as `NEW`, because a finding is identified by its headline (FR-009).
`ADDED`	The item exists only in the newer report **and** cites at least one source the older report did not cite. This is what distinguishes genuinely new *evidence* for a known point from a merely new *point*.
`UPDATE`	The item exists in both reports with changed condensed text - the same identity, drifted wording.
`OLD`	The item exists in both reports with unchanged condensed text.

"Same item" is decided by section-scoped normalised identity: findings are matched by their extracted headline, concepts by their declared name, and every other element by its normalised text with citations, Markdown emphasis, backticks and whitespace runs removed - so a cosmetically drifted item still matches itself and reports `UPDATE` rather than `NEW` + a silent deletion.

Slides The document is Markdown, separated into slides by `---` rules, in a fixed order:

1. **Cover** - the title, the generation date, the coverage window, the research id, and a table counting element slides, change items, and unchanged `OLD` items.
2. **One slide per concept**, then **one slide per other changed element** in a fixed element precedence: Executive Summary, Concepts, Findings, Cross-References, References.

   A **concept** is the deck's key category: the report's `## Concepts` section names the corpus's themes, and the deck renders each concept as its own slide carrying the concept's definition with the findings that belong to it grouped beneath it. Each concept slide is its name, its status, its definition, then one entry per finding:

   ```
   ## Human Oversight
   ```

```
_`UPDATE`_

Human oversight is the deliberate calibration of agent autonomy, with human
approval reserved for high-impact actions.

### `ADDED` Runtime enforcement beats paper approvals in agent governance

Paper approvals leave the gateway without context, so a policy document can
approve an agent the audit trail cannot reconstruct. Enforcement in the
request path generates audit evidence automatically at the moment of action.
```

A concept that claimed no finding still gets its slide and says so with `_No finding maps to this concept._`. Every finding no concept claimed is rendered on the **Other Findings** slide, so each finding appears on the deck exactly once and none is lost to the grouping.

The **other elements** (Executive Summary, Cross-References, References) each render every item of the element from both reports - the changed items *and* the unchanged `OLD` ones - as one condensed bullet labelled with its tag, so a slide is the complete picture of that element, not just its deltas:

```
## Cross-References

_NEW 1 - ADDED 0 - UPDATE 0 - OLD 1 - 2 item(s)_

- `OLD` `crates/ragent-llm/src/retry.rs` - backoff policy
- `NEW` `crates/ragent-agent/src/loop.rs` - turn driver
```

Item order within a slide follows the newer report, with the unchanged items kept in place; the count line is `NEW n - ADDED n - UPDATE n - OLD n - n item(s)`.

The report's `## Top 10 Implications` and `## Open Questions` are not deck elements: they are reader-facing narrative, and neither the slides nor the `--elements` filter names them.

3. **A `No changes` slide** instead of step 2, when the pair changed nothing at all. The deck is still written - an unchanged pair is a result, not an error.
4. **Summary** - the element-slide count, the per-status counts of every emitted item, and the names of the elements that produced a slide.

`--include-old` additionally emits a slide for an element that carries *only* `OLD` items, which is otherwise omitted.

Output file naming The deck is written to the item directory as `DECK-<YYYY-MM-DD>.md`, where the date is the UTC day the deck was *generated* - not a date from either contributing report. That is deliberate: regenerating a deck on a later day writes a new file beside the earlier one instead of overwriting it, so a series of decks can be kept just like a series of reports.

```
research/
  rust-async/
      RESEARCH-2026-02-14.md
      RESEARCH-2026-03-02.md
      DECK-2026-03-05.md       # generated 2026-03-05
      sources/
```

Write somewhere else with `--out <path>`; missing parent directories are created for you:

```
/research deck rust-async --out docs/decks/rust-async-diff.md
```

Re-running on the same day targets the same `DECK-<today>.md`. Without `--force` that is refused rather than overwritten, naming the existing path; `--force` overwrites it. A same-day rerun under `--force` is byte-identical.

Narrowing and flags Restrict the deck to specific elements with `--elements`, a comma-separated list of element names:

```
/research deck rust-async --elements findings,references
```

Names are matched case-insensitively and accept either the slide label or the report heading, so `"cross references"`, `"Cross-References"` and `"In-Project Cross-References"` all name the same element. An unknown name is refused with the offending value and the accepted list. The filter runs *before* rendering, so a narrowed deck's cover and summary counts describe only what it actually wrote.

Flag	Effect
`--from <YYYY-MM-DD>`	Older (baseline) report. Alone, pairs with the next newer report.
`--to <YYYY-MM-DD>`	Newer (current) report. Alone, pairs with the next older report.
`--elements <list>`	Comma-separated element names to include.
`--include-old`	Also emit slides for elements that changed nothing.
`--out <path>`	Write here instead of `DECK-<YYYY-MM-DD>.md`. Parent dirs created.
`--force`	Overwrite an existing deck file.

Refusals are non-destructive Every refusal exits before writing: two reports are read and parsed, the deck is rendered, *then* the destination is resolved. So a run that fails at any point leaves the item directory byte-identical - including any deck file already there. The refusals are:

- the item does not exist (no directory or file is created);
- the item holds fewer than two reports;
- the item holds only the legacy `RESEARCH.md`;
- a `--from` / `--to` value is not a real `YYYY-MM-DD` date, or names no existing report - the message lists the report dates that do exist;
- the pair is inverted (`--from` newer than `--to`);
- `--elements` names an unknown element;
- a selected report cannot be read, is not valid UTF-8, or contains no recognised top-level section - the message names the path and the reason, and `--force` does not bypass it;
- the target deck file exists and `--force` was not supplied.

CLI and event output The CLI prints one machine-readable event line and one human summary:

```
$ ragent research deck rust-async
ragent-research: {"kind":"deck","payload":{...}}
wrote `research/rust-async/DECK-2026-03-05.md` for `rust-async` (2026-02-14 to 2026-03-02): NEW 2
```

The payload's counts are the written deck's own, so the line can never disagree with the file on disk. A run that found no changes says `no changes detected between the two reports` rather than reporting a row of zeroes.

4A. Web-phase deadline

As of v1.0.100, `/research create` caps the web-gathering phase at 180 seconds by default. Use `--web-time N` (or `--web-phase-timeout-secs N`) to change the deadline; set `--web-time 0` to disable it.

When the deadline elapses, the run ingests everything gathered so far and continues to analysis/synthesis with the partial source set. The deadline bounds the *search stage* only: no new search is started after the deadline. The fetch stage is never gated on or cancelled by the deadline — every candidate found by a truncated search stage is fetched to completion, and each fetch is individually capped by `--fetch-timeout-secs` (default 30 s).

The TUI status bar shows a live `web:M:SS` countdown while the web phase is active, and a single quantified notice is added to the research progress message when the deadline is reached, for example:

```
**Web phase deadline reached** - 7 source(s) captured before timeout; proceeding to synthesis
```

---

5. Tiers: light, full, dissertation

The `--tier` flag selects how much adversarial quality assurance runs after gathering. The default is `full`.

Tier	Pipeline steps	Best for	Min sources (vault skip)
`light`	decompose, width sweep, evidence digest, triple draft, synthesize, cite check, polish	Quick orientation, fast answers	3
`full`	All 16 Hyperresearch steps (see section 12)	Thorough reports, decision documents	8
`dissertation`	`full` plus chapter partitioning into up to 12 chapters	Long-form deliverables	15

When to use each tier

- `light` — when you need a quick answer and do not need contradiction analysis or deep evidence auditing. Runs in seconds to a minute.
- `full` (default) — when you need a thorough, decision-quality report with contradiction detection, evidence ranking, and citation verification. Runs in minutes.
- `dissertation` — when you need a long-form, chaptered deliverable suitable for extended writing. The pipeline partitions the topic into up to 12 chapters and runs the full adversarial pipeline per chapter.

```
/research create quick "What is WebAssembly" --tier light
/research create decision "Rust vs Go for microservices" --tier full
/research create survey "History of functional programming" --tier dissertation
```

---

6. Depth and iterations

`--depth` selects source and iteration budgets. The default is `standard`.

Depth	Iterations	Sources per sub-question	Concurrency
`shallow`	1	2	2
`standard` (default)	3	3	4
`deep`	5	5	6

Web volume is bounded by depth by default. Unless `--max-web-results` is passed explicitly, the effective web-source budget for gather passes is derived from the selected depth (shallow 6 / standard 9 / deep 15 sources), so `--depth shallow` actually limits search/fetch volume instead of collapsing into the 500-source ceiling. `--max-search-calls N` additionally places a hard, run-scoped cap on the total number of web-search calls a run may issue, shared across every supervisor/competitive researcher and gather pass; when the cap is reached, remaining sub-queries are skipped and the run proceeds with the sources gathered so far.

Iteration override

`--iterations N` overrides the default iteration count. When set (or when `--depth deep` is used), the iterative research engine activates and runs a planner/critic loop:

1. The **planner** decomposes the topic into sub-questions (using the heuristic planner by default, or the LLM planner when available).
2. Each iteration gathers sources for pending sub-questions concurrently (up to `max_concurrency`).
3. The **critic** evaluates the gathered evidence, scores quality, and identifies evidence gaps.
4. Gaps generate follow-up queries for the next iteration.
5. An **adaptive stopper** decides whether to continue: it stops when the critic score is high enough, all sub-questions are answered, or the maximum iteration count is reached.
6. If no progress is made (no new sources, no answered questions) and `force_deeper` is not set, the loop stops early.

Local source budgets

The depth also controls how many local files are captured:

Depth	Max local sources
`shallow`	5
`standard`	10
`deep`	20

```
/research create fast "Quick overview of REST APIs" --depth shallow
/research create balanced "REST vs GraphQL vs gRPC" --depth standard
/research create thorough "Comprehensive guide to API design" --depth deep --iterations 8
```

---

7. Output formats

`--format` changes the skeleton and final layout of the report.

Format	Layout
`report` (default)	Topic, Search Queries, Executive Summary, Top 10 Implications, Open Questions, Findings, Findings Relationship Diagram, Cross-References, References Index
`executive-summary`	One-page summary of the same content
`comparison-table`	Topic, Research Brief, Executive Summary, Comparison Criteria, Comparison Table, Entity Profiles, Findings, References Index
`source-bibliography`	Bibliography of all captured sources
`imrad`	Scientific/technical report: Abstract, Introduction, Methods, Results, Discussion, References Index
`newsletter`	Reader-facing subset only: Topic, Executive Summary, Top 10 Implications, Open Questions, Concepts, Findings, Findings Relationship Diagram

The format is recorded in the item frontmatter and can be reopened later with `/research open`.

CORPA companion

Quality-assurance renders (Contradiction Graph, Loci Analysis, Depth Investigation, Cross-Locus Reconcile, Source Tensions, Synthesis Audit, Corpus Critic) are written to `research/<name>/CORPA-<YYYY-MM-DD>.md` rather than inline in the report. The companion file is created with the skeleton and rewritten on every full write; its `## Sources Reference` section repeats the References Index table so `[#N]` indices resolve in either document.

Format aliases

The parser accepts several aliases for each format:

Format	Accepted values
`report`	`report`, `default`
`executive-summary`	`executive-summary`, `executive_summary`, `summary`
`comparison-table`	`comparison-table`, `comparison_table`, `comparison`
`source-bibliography`	`source-bibliography`, `source_bibliography`, `bibliography`
`imrad`	`imrad`, `im-rad`, `scientific`
`newsletter`	`newsletter` (no aliases)

Format	Accepted values

When to use each format

- `report` — general-purpose research report with findings and implications. The default for most use cases.
- `executive-summary` — when you need a one-page brief for decision-makers.
- `comparison-table` — when evaluating options (tools, frameworks, approaches) side by side.
- `source-bibliography` — when you need a clean bibliography of all sources without narrative findings.
- `imrad` — for scientific or technical reports following the Introduction/Methods/Results/Discussion structure.
- `newsletter` — when the artifact is meant to be read rather than audited: a lead paragraph, the ten ranked implications, the open questions, the concepts and the findings, with the corpus scoreboard, research brief, search-query/engine blocks, data-quality sections, cross-references and references index all omitted.

```
/research create exec-brief "Q4 security posture" --format executive-summary --tier light
/research create tool-comparison "PostgreSQL vs MySQL vs SQLite" --format comparison-table
/research create bib "Sources on transformer architectures" --format source-bibliography
/research create sci-report "Reproducing GPT-3 results" --format imrad --tier full
/research create briefing "AI economics in late 2026" --format newsletter --tier light
```

---

8. Seed sources and local context

8.1 Seed a research from a URL

```
/research create from-url --from-url https://example.com/article
```

The page is fetched first; its extracted body is used to derive the topic, then normal web search runs on that topic. The fetched page is also captured as the first primary web source. `--from-url` can be repeated for multiple seed pages:

```
/research create multi-seed --from-url https://blog.example.com/post1 --from-url https://blog.exa
```

When a topic is also supplied, both the topic and the fetched page content are used. When no topic is supplied, the topic is derived from the cleaned page body (via readability extraction), falling back to the page title, then the URL.

8.2 Seed from a local file

```
/research create from-doc --from-file docs/design.md --use-local
```

The file body is extracted as the primary `Other` source and used to derive the topic. Supported file formats:

- PDF (`.pdf`)
- Plain text (`.txt`)
- Markdown (`.md`)

When no topic is supplied, a concise topic is derived from the extracted text. The normal web-search phase still runs using the derived topic.

```
/research create from-pdf --from-file papers/attention-is-all-you-need.pdf --format imrad --tier
/research create from-doc --from-file docs/design.md --use-specs
```

8.3 Include project files and specs

```
/research create rust-async "Rust async patterns" --use-local --use-specs
```

- `--use-local` enables scanning the project root and any `--sources-dir` for files matching the topic keywords. Files are scored by keyword overlap and the top results are captured with excerpts.
- `--use-specs` cross-references prior specifications under `specs/<id>/`. Each spec's SPEC.md is scanned for relevance and captured as a spec source.

8.4 Extra sources directory

```
/research create with-extras "Project architecture" --use-local --sources-dir docs/architecture
```

`--sources-dir <path>` adds an additional directory to the local scan. Files in this directory are scored alongside project-root files.

---

9. Tuning gathering behavior

These flags control the performance and resilience of the gathering phases.

Flag	Default	What it does
`--fetch-concurrently N`	10	Number of candidate pages fetched in parallel during web gathering. `0` is clamped to `1`.
`--local-concurrently N`	8	Parallel local-file scoring tasks. `0` is clamped to `1`.
`--fetch-timeout-secs N`	30	Per-page fetch timeout. Pages exceeding this are treated as fetch failures.
`--web-time N`	180	Wall-clock timeout for the web *search stage* (alias of `--web-phase-timeout-secs`). When the deadline passes, everything gathered so far is ingested and the run continues to analysis/synthesis with the partial source set. `0` disables the deadline. No new search is started after the deadline; fetches are never gated on or cancelled by it — each fetch is individually capped by `--fetch-timeout-secs`.
`--local-phase-timeout-secs N`	none	Wall-clock timeout for the entire local phase. Aborts if exceeded.
`--search-max-retries N`	2	Retries per failed sub-query. `0` disables retries.
`--search-retry-base-delay-ms N`	200	First retry delay in ms, doubled each retry (200, 400, 800…).

Flag	Default	What it does
`--max-search-calls N`	derived from `--depth`	Hard, run-scoped cap on total web-search calls, shared via `Arc` across every supervisor/competitive researcher and gather pass. A run-scoped query cache memoises identical sub-queries so parallel researchers reuse cached hits instead of re-issuing paid calls.
`--max-web-results N`	derived from `--depth` (shallow 6 / standard 9 / deep 15)	Cap on the web-source budget.
`--max-queries N`	`research.max_queries` (20)	Cap on the number of search queries the topic-decomposition step may produce. The cap bounds the model request, the heuristic fallback, and the queries the gatherer actually searches. `0` means unbounded (still subject to the decomposer's hard upper bound of 20).
`--max-concepts N`	`research.max_concepts` (5)	Output cap on the `## Concepts` list. Entries are ordered most-relevant-first before truncation. `0` means unbounded.
`--max-findings N`	`research.max_findings` (20)	Output cap on the `## Findings` list. Entries are ordered most-relevant-first before truncation. `0` means unbounded.
`--use-low-relevance`	off	Keep sources that would normally be filtered out as low-relevance. The pre-fetch filter matches query terms morphologically (plurals, gerunds, and derived forms such as "agentic" matching "agent" or "loops" matching "loop") and retains hits scoring Medium (35%+ term overlap) or better.
`--papers`	off	Include academically-classified backends (OpenAlex) in the search sweep. Scholarly engines are excluded by default, so this flag opts them back in for academic topics.
`--use-pdf`	off	Allow PDF documents from web search or `--from-url` to be captured as sources.
`--oa-enable`	`research.open_access_recovery` (off)	Force open-access recovery on for this run, overriding `ragent.json`.
`--no-oa`	`research.open_access_recovery` (off)	Force open-access recovery off for this run, overriding `ragent.json`.
`--url-cloak`	off	Emit web source URLs as defanged plain text (`hxxps://host[.]tld/...` in a code span) in the `Sources` bullets and the `References Index` table, so automated URL scanners do not flag the document. Non-URL rows are unchanged.

Flag	Default	What it does
`--age <span>`	off	Reject web sources published longer ago than the span (`30d`, `6m`, `2y`); a source with no publication date is rejected too. The window's start date (`today - span`, rendered as an ISO `YYYY-MM-DD`) is appended to every decomposed search query, so the engines steer towards material inside the window rather than the run discarding an all-old result set after fetching it.

GitHub `blob/` file-view URLs (`github.com/owner/repo/blob/...`) are rewritten to `raw.githubusercontent.com` before fetching and non-HTML content bypasses the readability gate, so GitHub file URLs no longer fail gathering with a readability error.

Performance tuning examples

For a slow network, reduce concurrency and increase timeouts (including the 180-second web phase deadline):

```
/research create slow-net "Distributed systems" --fetch-concurrently 4 --fetch-timeout-secs 60 --
```

For a fast network with many expected results, increase concurrency:

```
/research create fast-net "Rust ecosystem 2024" --fetch-concurrently 20
```

To prevent a slow local filesystem scan from blocking, set a local phase timeout:

```
/research create big-project "Codebase architecture" --use-local --local-phase-timeout-secs 120 -
```

To make search more resilient against transient failures:

```
/research create resilient "Edge computing platforms" --search-max-retries 5 --search-retry-base-
```

---

10. The Hyperresearch pipeline in `full` tier

The `full` (and `dissertation`) tiers run an adversarial, deterministic quality pipeline after gathering. The pipeline has 16 steps, each of which produces observable progress events:

1. **Decompose** — break the topic into sub-queries using the planner.
2. **Width sweep** — parallel web search across all sub-queries.
3. **Contradiction graph** — detect opposing source claims on the same dimension and rank them by strength. The graph uses polarity dimensions (e.g. positive vs negative sentiment) to identify pairs of sources that make contradictory claims. Each contradiction is scored: base 30 + 20 per overlapping dimension, capped at 100.
4. **Loci analysis** — find recurring research dimensions (loci) across the corpus. Each locus represents a key claim or dimension that multiple sources address.
5. **Depth investigation** — classify each locus as `surface`, `moderate`, or `deep` based on how many sources address it.

6. **Cross-locus reconcile** — surface dimensions that share common sources, identifying where evidence overlaps and where it diverges.
7. **Source tensions** — list contradictions, shallow evidence, and isolated sources that lack corroboration.
8. **Corpus critic** — audit evidence quality and coverage. Identifies sub-questions with no sources, weak evidence, and coverage gaps.
9. **Evidence digest** — rank claims from strongest to weakest support, marking contested ones. Strong claims have multiple independent sources; weak claims rely on a single source or face contradictions.
10. **Triple draft** — produce three deterministic candidate summaries:
    - **Consensus** — the majority view across sources.
    - **Skeptical** — a cautious view that flags contested claims.
    - **Gap-aware** — highlights what the evidence does not cover.
11. **Synthesize** — combine the triple drafts into a final narrative draft, weighted by evidence strength and critic scores.
12. **Critics** — four internal critic subagents score the draft on:
    - **Coverage** — are all sub-questions addressed?
    - **Logic** — do findings follow from evidence?
    - **Evidence** — are claims properly supported?
    - **Readability** — is the draft clear and well-structured?
13. **Gap-fill fetch** — issue targeted follow-up queries for evidence gaps identified by the corpus critic. New sources are fetched and added to the corpus.
14. **Patcher** — apply deterministic surgical revisions to the draft based on audit and critic output. Fixes citation mismatches, clarifies contested claims, and strengthens weak evidence.
15. **Cite check** — verify that every `[#N]` citation marker in the draft points to a gathered source. A failure blocks report shipment and emits a `CITATION_VERIFICATION_FAILED` diagnostic.
16. **Polish** — clean whitespace and control characters, finalise formatting, and score the final draft.

The `dissertation` tier adds a **ChapterPartition** step before all others, dividing the topic into up to 12 chapters. Each chapter then runs the full pipeline independently.

Each step appears in the report when it produced results. Steps that produce no output (e.g. no contradictions found) are omitted from the final report. The run manifest tracks every step's lifecycle (pending, in-progress, completed, skipped, failed) and is persisted for resumability.

---

11. Reading results in the TUI

`/research open <name>` opens the markdown viewer overlay.

Key / mouse	Action
`Up` / `Down`	Scroll one line
`PageUp` / `PageDown`	Scroll one page
`Ctrl+PageUp` / `Ctrl+PageDown`	Jump to start / end
`Mouse wheel`	Scroll inside the overlay when the cursor is over it
`Click outside` or `Esc`	Close the overlay

The viewer renders headings, lists, code blocks, tables, and inline formatting. Tables (including the references index) are preserved as rows.

12. Following live progress

When a research session runs, the TUI shows progress in three places:

1. **Status bar** — a transient line such as `research: rust-async -- web -- running`. It updates per phase and becomes `research: rust-async complete -- 12 sources` when finished. While the web phase is active, the top-right status segment also shows a live `web:M:SS` countdown from the phase deadline (see below).
2. **Message window** — a pinned `Research Progress -- rust-async` log lists each phase (setup, web, local, specs, synthesize, assemble, finalize) and audit results. The web phase ends with a per-engine width-sweep table (separated from the surrounding lines by a blank line), showing for each backend search engine how many candidates were `considered`, `captured`, and `excluded`, plus a breakdown of the exclusion reasons: `papers` (scholarly-engine blocks - clear with `--papers`), `pdf` (disabled PDF sources), `relev` (low title/snippet or post-fetch relevance), `short` (content below the minimum extractable length), and `fetch` (page fetch failures/timeouts). The `fetch` column is broken down further into the cause of each failed fetch: `t/o` (per-fetch timeout), `net` (transport/network error), `blk` (blocked before the request by the SSRF guard or robots.txt), `http` (server error status), `wall` (paywall/auth wall), `js` (JavaScript-rendered shell), and `extr` (content retrieved but extraction failed). Every per-engine row and the `totals` row carry the reason columns and the fetch-failure columns; the totals reasons sum back to the reported `excluded` count, and the totals fetch-failure counts sum back to the totals `fetch` count.
3. **Log panel** — every progress event is logged at `Info` level.

Research runs in the background; you can continue typing or start other work while it gathers.

Web-phase deadline countdown and timeout notice

When a `/research create` run enters its web-gathering phase, the TUI status bar shows a live countdown in the top-right wait segment, for example `web:0:45`. The countdown is computed from a stored wall-clock deadline at render time, so it stays accurate even if the event loop skips redraws or receives a burst of events (FR-010, FR-013). It updates at least once per second while the web phase is active and is removed as soon as the phase completes, times out, or fails (FR-011).

When the configured `--web-time` deadline elapses, the run continues with the sources already captured. A single, quantified notice is added to the research progress message, for example:

```
**Web phase deadline reached** - 7 source(s) captured before timeout; proceeding to synthesis
```

This notice is rendered at most once per run (FR-012). The underlying `PhaseTimedOut` diagnostic is also emitted exactly once per gather phase, carrying the effective deadline and the final captured count (FR-004).

Set `--web-time 0` to disable the deadline and allow the web phase to run to natural completion (FR-007).

Progress events

The session emits structured events that the TUI renders:

- **Phase** — phase transitions (setup, web, local, specs, synthesize, assemble, finalize).
- **QueriesDecomposed** — the sub-queries generated by the planner.

- **WebCaptured** — each web source captured (URL, title, search engine, body preview, language, media type, OA recovery info). The TUI folds these into the per-engine capture summary table; the log panel keeps a one-line-per-URL record.
- **FromUrlBodyPreview** — preview of a `--from-url` seed page body.
- **FromFileBodyPreview** — preview of a `--from-file` seed document body.
- **LocalCaptured** — each local file captured (path, relevance score).
- **SpecCaptured** — each spec captured (spec ID).
- **IterationCompleted** — iteration number and critic score.
- **CriticResult** — critic quality score and identified evidence gaps.
- **VerificationResult** — citation verification pass/fail and issues.
- **FollowUpQueries** — gap-driven follow-up queries for the next iteration.

---

13. Web search engines

By default, research uses `mf_search`, which queries OpenAlex and Wikipedia in parallel. Optional API-backed engines can be added in `ragent.json`:

```
{
    "langsearch_api_key": "...",
    "tavily_api_key": "...",
    "perplexity_api_key": "...",
    "exa_api_key": "...",
    "serper_api_key": "...",
    "openalex_email": "you@example.com"
}
```

Use the TUI diagnostics to check availability:

```
/websearch show   # enabled / in-use / failed per engine
/websearch test   # live probe each configured engine
```

The `--papers` flag includes academically-classified backends (OpenAlex) in the search sweep. Because scholarly engines are excluded by default, this flag opts them back in for academic topics where scholarly results help. Non-academic engines (Wikipedia and any configured web engines) always run. Set `research.exclude_academic_engines: true` in `ragent.json` to make the exclusion explicit; a per-run `--papers` overrides it, and the flag wins when both are present.

Engine exclusion

Under the hood the scholarly-engine default and `research.exclude_academic_engines` map onto the `mf_search` `exclude_engines` parameter (an array of engine names) and the research `WebSearchTool` exclusion argument. Exclusions are applied to the orchestrator *before* any request is dispatched, so excluded engines are never queried. Names that do not match a registered engine are ignored. When the exclusion list names every configured engine, `mf_search` returns an explicit "all engines excluded" result (with `excluded_engines` in the metadata) rather than a silent zero-result success.

Merge diversity

When two or more engines return results, the consensus merge caps any single engine at half of the merged slots per sub-query. An engine that returns a large scored block (e.g. OpenAlex) cannot crowd the other

engines out of the gathered source list; slots the other engines cannot fill are handed back to the dominant engine's next-best results. With only one contributing engine the merge keeps its full budget.

Fetch budget

By default the width sweep runs **uncapped**: every unique candidate URL that passes the relevance pre-filter is fetched, and every sub-query search asks the engines for their maximum result page (e.g. OpenAlex returns up to 75 results per query). Volume is bounded by the per-fetch timeout (`--fetch-timeout-secs`) and the fetch concurrency (`--fetch-concurrently`), not by a hard cap — a separate cap would only discard candidates the search already paid for. `capped` in the width-sweep summary is therefore 0 on normal runs.

Competitive (comparison) runs (`--mode competitive`) cap the volume: each sub-query search asks for at most the web budget (`--max-web-results`, or the depth-derived budget when unset) hits, and the fetch budget is that cap multiplied by the number of sub-queries; anything beyond it is reported as `capped` in the width-sweep summary. This keeps one comparison from issuing engine-max searches per entity.

The relevance pre-filter also rescues on-topic results whose titles match two or more query terms even when the overall term ratio falls below the Medium floor — verbose decomposed sub-queries ("how to write goals and configure AI agent loops") otherwise reject good hits like "Designing agentic loops" purely because the query has many terms.

Keyless backends

These backends require no API keys and run by default:

- **OpenAlex** — scholarly works catalog (set `openalex_email` for the polite pool)
- **Wikipedia** — English Wikipedia encyclopedia summaries

API-backed backends

These require API keys configured in `ragent.json`:

- **LangSearch** — `langsearch_api_key`
- **Tavily** — `tavily_api_key`
- **Perplexity** — `perplexity_api_key`
- **Exa** — `exa_api_key`
- **Serper** — `serper_api_key` (or the `SERPER_API_KEY` environment variable)

---

14. Open-access recovery

For scholarly sources, ragent can attempt to recover a legal full-text copy via Unpaywall and Europe PMC when the fetched body is shorter than the configured minimum (default: 1000 characters).

Enable it in `ragent.json`:

```
{
    "research": {
        "open_access_recovery": true,
        "contact_email": "you@example.com",
        "oa_min_full_text_chars": 1000
```

```
    }
}
```

Or turn it on for a single run with `--oa-enable` (and off with `--no-oa`) without editing the config. Precedence: an explicit `--oa-enable`/`--no-oa` wins, otherwise `research.open_access_recovery` decides, and when neither is set recovery is off.

`contact_email` is required by Unpaywall's terms of service. When a source is recovered from an OA copy, the references index notes the source URL, OA source (Unpaywall or Europe PMC), version, and license.

The recovery process:

1. The web gatherer fetches a scholarly source page.
2. If the body is shorter than `oa_min_full_text_chars`, the gatherer queries Unpaywall for an OA copy using the source URL.
3. If Unpaywall returns a result, the OA copy is fetched.
4. If Unpaywall does not have a copy, Europe PMC is queried as a fallback.
5. The recovered full text replaces the short body, and the source's metadata records the OA recovery provenance.

---

15. Source vault and resumability

The source vault (`.ragent/research_vault/<run_tag>/`) persists raw captured source text and metadata in a SQLite index. Each vault entry records:

- Source URL, title, and fetch timestamp
- Search tool and engine that discovered the source
- Media type classifier (`page`, `pdf`, `youtube`, etc.)
- On-disk path to the raw content file
- Blake3 hash of the body text
- Full body text for full-text search

On a subsequent run with the same research name, the gatherer checks the vault before issuing new web searches. If the vault already contains enough sources to satisfy the requested tier's minimum, web search is skipped entirely. This avoids re-fetching already-captured sources and makes re-runs nearly instant.

Vault skip thresholds

Tier	Minimum sources to skip web search
`light`	3
`full`	8
`dissertation`	15

Run manifest

Each run also persists a **run manifest** (JSON) that tracks every pipeline step's lifecycle. On resume, the tier router reads the manifest and skips already-completed steps, restarting from the first pending or in-progress step. This means a interrupted `full`-tier run can be resumed without re-running earlier pipeline phases.

---

16. Templates

Place Markdown templates under `research/_templates/<name>.md`. They can use the placeholders `{{title}}`, `{{topic}}`, and `{{date}}`.

```
/research create quarterly "Q3 performance review" --template quarterly
```

The template becomes the skeleton and the standard sections are appended below it. Template placeholders are substituted:

- `{{title}}` — the research title (derived from the topic or seed source)
- `{{topic}}` — the research topic string
- `{{date}}` — the current date in ISO format

Example template

`research/_templates/quarterly.md`:

```
# {{title}}

**Period:** {{date}}

## Executive Overview

## Key Metrics

## Goals for Next Quarter
```

When used with `/research create quarterly "Q3 performance review" --template quarterly`, the template provides the skeleton and the standard research sections (findings, implications, references, etc.) are appended below the template content.

---

17. HTTP API

The research system is also accessible via HTTP endpoints. All endpoints are mounted under the auth-protected router.

GET /research

List every research item (excludes archived by default).

Response:

```
{
    "items": [
        {
            "name": "rust-async",
            "title": "Rust async/await patterns",
            "status": "complete",
```

```
            "created_at": "2024-01-15T10:00:00Z",
            "modified_at": "2024-01-15T10:05:00Z",
            "sources": 12
        }
    ],
    "count": 1
}
```

POST /research

Create and run a research session. Returns the research name, format, total source count, and all session events as a JSON array.

Request body:

```
{
    "name": "rust-async",
    "topic": "Rust async/await patterns",
    "title": "Optional custom title",
    "from_urls": ["https://example.com/article"],
    "from_file": "docs/design.md",
    "sources_dir": "docs/architecture",
    "template": "quarterly",
    "use_local": true,
    "use_specs": true,
    "use_low_relevance": false,
    "papers": false,
    "use_pdf": false,
    "fetch_concurrency": 10,
    "fetch_timeout_secs": 30,
    "local_concurrency": 8,
    "depth": "standard",
    "iterations": 3,
    "max_concepts": 5,
    "max_findings": 20,
    "max_queries": 20,
    "url_cloak": false,
    "format": "report"
}
```

Response (201 Created):

```
{
    "name": "rust-async",
    "format": "report",
    "total_sources": 12,
    "events": [
        { "type": "phase", "phase": "setup" },
        { "type": "queries", "queries": ["..."] },
        { "type": "web", "url": "...", "title": "..." }
    ]
```

```
}
```

Error codes:

- `400 Bad Request` — invalid research name
- `409 Conflict` — research item already exists
- `500 Internal Server Error` — gathering or synthesis failure

GET /research/{name}

Show metadata for a single item, including related research items (top 5 by title similarity).

Response:

```
{
    "item": {
        "name": "rust-async",
        "title": "Rust async/await patterns",
        "status": "complete",
        "created_at": "2024-01-15T10:00:00Z",
        "modified_at": "2024-01-15T10:05:00Z",
        "sources": 12
    },
    "related": [
        {
            "name": "tokio-runtime",
            "title": "Tokio runtime internals",
            "snippet": "..."
        }
    ]
}
```

Error codes:

- `404 Not Found` — item not found (includes closest matches)
- `400 Bad Request` — invalid name

PUT /research/{name}

Replay the item's recorded invocation line: re-runs the full pipeline and writes a new dated report and its associated files with freshly gathered results. This is the HTTP equivalent of `/research update <name>` (and `ragent research update <name>`). The response uses the same envelope as `POST /research`.

```
curl -s -X PUT http://localhost:9100/research/rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

Error codes:

- `404 Not Found` — item not found
- `409 Conflict` — a run for this item is already in flight

DELETE /research/{name}

Delete a research item. Requires a confirmation token.

Query parameter: `?confirm=delete-{name}`

The examples below assume a server started with `ragent serve --addr 127.0.0.1:9100` (the default bind address is `127.0.0.1:3000`).

```
curl -X DELETE http://localhost:9100/research/rust-async?confirm=delete-rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

Error codes:

- `412 Precondition Failed` — missing or incorrect confirm token
- `404 Not Found` — item not found

Example: full HTTP workflow

```
# Create a research item
curl -s -X POST http://localhost:9100/research \
  -H "Authorization: Bearer $RAGENT_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"rust-async","topic":"Rust async/await patterns","use_local":true}' \
  | jq '.name, .total_sources'

# List all items
curl -s http://localhost:9100/research \
  -H "Authorization: Bearer $RAGENT_TOKEN" | jq '.items[] | .name'

# Show one item
curl -s http://localhost:9100/research/rust-async \
  -H "Authorization: Bearer $RAGENT_TOKEN" | jq '.item.title'

# Delete an item
curl -s -X DELETE "http://localhost:9100/research/rust-async?confirm=delete-rust-async" \
  -H "Authorization: Bearer $RAGENT_TOKEN"
```

---

18. CLI equivalents

The same commands work outside the TUI:

```
ragent research help
ragent research create rust-async "Rust async patterns" --tier full --use-local
ragent research create brief "Vector databases" --max-concepts 3 --max-findings 10
ragent research create vendors "Managed vector db vendors" --url-cloak
ragent research create from-url --from-url https://example.com/article
ragent research create from-doc --from-file docs/design.md --use-local
ragent research create comp "vector db landscape" --mode competitive
ragent research open rust-async
ragent research list
ragent research list --all
ragent research search "vector database"
ragent research show rust-async
```

```
ragent research update rust-async
ragent research delete rust-async --yes
ragent research archive rust-async
ragent research continue rust-async "Also cover tokio vs async-std"
ragent research deck rust-async
ragent research deck rust-async --from 2026-02-14 --to 2026-03-02
ragent research deck rust-async --elements findings,open-questions --include-old
ragent research deck rust-async --out docs/decks/rust-async-diff.md --force
```

CLI `create` emits machine-readable JSON lines prefixed with `ragent-research:` so you can pipe them through `jq`:

```
ragent research create rust-async "Rust async patterns" 2>&1 | grep '^ragent-research:' | jq '.'
```

Each JSON line is a session event (phase, queries, web source captured, local source captured, iteration completed, critic result, etc.).

CLI `deck` emits exactly one `ragent-research: {"kind":"deck","payload":{...}}` line per successful run, followed by a human summary:

```
ragent research deck rust-async | grep '^ragent-research:' | jq '.payload'
```

The payload carries `name`, `path`, `from`, `to`, `no_changes`, the per-status counts (`new`, `added`, `update`, `old`, `changes`), and `element_slides` - the element labels that produced a slide. See § 4.12 for the full deck contract.

---

19. Configuration

Research-specific configuration lives under the `research` key in `ragent.json`:

```
{
    "research": {
        "open_access_recovery": true,
        "exclude_academic_engines": false,
        "contact_email": "you@example.com",
        "oa_min_full_text_chars": 1000,
        "max_concepts": 5,
        "max_findings": 20,
        "max_queries": 20,
        "evaluate": {
            "enabled": true
        }
    }
}
```

Field	Type	Default	Description
`open_access_recovery`	`bool`	`false`	Enable OA recovery via Unpaywall and Europe PMC

Field	Type	Default	Description
`exclude_academic_engines`	bool	`false`	Keep academically-classified engines (OpenAlex) excluded from research runs. Scholarly engines are already excluded by default; a per-run `--papers` opts them back in and takes precedence.
`contact_email`	string?	`null`	Email required by Unpaywall's ToS
`oa_min_full_text_chars`	usize	`1000`	Minimum body length that triggers OA recovery
`max_concepts`	usize	`5`	Output cap on the `## Concepts` list. A per-run `--max-concepts` takes precedence; `0` means unbounded.
`max_findings`	usize	`20`	Output cap on the `## Findings` list. A per-run `--max-findings` takes precedence; `0` means unbounded.
`max_queries`	usize	`20`	Cap on the number of search queries produced by topic decomposition. A per-run `--max-queries` takes precedence; `0` means unbounded.
`evaluate`	ResearchEvaluateConfig	`{"enabled": false}`	Self-evaluation scorecard settings (FR-015 of specs/opendeepresearch). When enabled, the pipeline appends a deterministic quality scorecard (quality, relevance, groundedness, completeness, structure) to the report.

Web search engine keys are top-level in `ragent.json`:

```
{
    "langsearch_api_key": "...",
    "tavily_api_key": "...",
    "perplexity_api_key": "...",
    "exa_api_key": "...",
    "serper_api_key": "...",
    "openalex_email": "you@example.com"
}
```

20. Tips for good results

- **Use specific topics.** Broad topics produce broad findings. Narrow questions yield more actionable reports. “Rust async cancellation semantics” is better than “Rust async”.
- **Pick the right tier.** `light` for quick lookups, `full` for decision documents, `dissertation` for long-form writing.
- **Pick the right depth.** `shallow` for fast orientation, `standard` for balanced research, `deep` for thorough investigation with more sources.
- **Enable `--use-local` and `--use-specs`** when the research should tie back to your own codebase or specifications.
- **Seed with `--from-url` or `--from-file`** when you want the report to start from a particular document.
- **Review the contradiction graph and source tensions** in `full` tier reports to spot conflicting evidence before acting on the findings.
- **Check `/websearch test`** if a run returns unexpectedly few web sources — a backend may be rate-limited or blocked.
- **Read the progress log** to see which sources were captured, which failed, and whether the synthesis used the LLM or the mechanical fallback.
- **Use `--papers`** for academic topics where scholarly results are wanted; they are excluded by default.
- **Use `--use-pdf`** when PDFs are likely to contain the best evidence (white papers, technical reports, academic papers).
- **Use `--use-low-relevance`** when researching niche topics where even low-relevance sources may contain useful information.
- **Use `--web-time`** to raise (or lower) the web phase deadline — by default the web phase is capped at 180 seconds, after which everything gathered so far is ingested and the run continues to analysis/synthesis. Set `--web-time 0` to remove the cap on slow networks. Once the deadline elapses, no new search is started; every candidate found is still fetched to completion (each capped by `--fetch-timeout-secs`, default 30 s), so the phase always returns.
- **Re-run with the same name** to leverage the source vault — if enough sources are already cached, web search is skipped and the run is nearly instant.
- **Use templates** for recurring report formats (quarterly reviews, security assessments, etc.) to ensure consistent structure.
- **Use `--format comparison-table`** when evaluating competing options — the comparison table layout makes trade-offs visible at a glance.
- **Use `--format imrad`** for scientific or technical reports that need the Introduction/Methods/Results/Discussion structure.

---

21. Troubleshooting

Symptom	Likely cause	What to do
`0 sources` or very few	Search backends blocked / rate-limited	Run `/websearch test`; wait or add an API-backed engine
`LLM synthesis failed - using mechanical fallback`	No API key, model down, or output malformed	Check provider setup; the report still contains deterministic summaries
`CITATION_VERIFICATION_FAILED`	A citation marker `[#N]` points to a missing source	Run again with `--tier full`; the gate blocks shipment

Symptom	Likely cause	What to do
`web phase deadline reached`	The 180-second web phase deadline passed (slow network or many candidate pages)	The run continues with the sources gathered so far; raise `--web-time N` or lower `--fetch-concurrently` for fuller coverage. The deadline stops new searches from being started (search stage only); fetches already queued always finish, each capped by `--fetch-timeout-secs`.
`local phase timed out`	Large project with many files	Raise `--local-phase-timeout-secs` or lower `--local-concurrently`
Name rejected	Invalid research name	Use 3–64 lowercase ASCII letters/digits/hyphens starting with a letter
`research/<name>` already exists	Duplicate create	Use `/research open <name>` or pick a new name
Very slow run	Deep depth + full tier + many sources	Use `--depth standard` or `--tier light`; reduce `--fetch-concurrently`
No scholarly sources	Scholarly engines are excluded by default	Pass `--papers` to include them; check `/websearch test` for OpenAlex status
OA recovery not working	Not enabled or `contact_email` missing	Pass `--oa-enable`, or set `research.open_access_recovery: true` (plus `contact_email`) in `ragent.json`
PDFs skipped	`--use-pdf` not set	Add `--use-pdf` to allow PDF web sources
Sources not reused on re-run	Vault threshold not met	The vault needs 3/8/15 sources (light/full/dissertation) before skipping web search

---

22. End-to-end examples

Example 1: Technology evaluation

Research and compare three vector databases for a RAG system:

```
/research create vector-db-eval "Comparison of pgvector, Qdrant, and Weaviate for RAG application
```

This produces a comparison-table report with each database evaluated across multiple dimensions (performance, scalability, ease of use, ecosystem), gathered from web sources and cross-referenced with any local project files.

Example 2: Academic paper review

Research a specific paper using the IMRaD format:

```
/research create attention-paper "Attention Is All You Need - Transformer architecture analysis"
```

This seeds the research from the arxiv page, derives the topic from the abstract, gathers related sources, and produces an IMRaD-structured report.

Example 3: Project architecture audit

Audit your own codebase architecture with web context:

```
/research create arch-audit "Analysis of current project architecture patterns and improvement op
```

This scans project files under `src/`, cross-references existing specs, and supplements with web research on architecture best practices.

Example 4: Quick competitive intelligence

Fast check on a competitor's latest release:

```
/research create competitor-check "Latest features in Kubernetes 1.30" --tier light --depth shall
```

This produces a quick summary without the full adversarial pipeline.

Example 5: Long-form survey

Comprehensive survey for a thesis or white paper:

```
/research create multi-agent-survey "Survey of multi-agent orchestration frameworks and patterns"
```

This partitions the topic into chapters, runs the full pipeline per chapter, and produces a long-form deliverable.

Example 6: Seeded from a local design document

Research starting from a local design doc, augmented with web sources:

```
/research create design-review --from-file docs/architecture/design.md --use-local --use-specs --
```

The design document is extracted as the primary source, the topic is derived from its content, and web search supplements with external context.

End of manual.